

PYTHON PARA IA

El motor detrás de la revolución de los Agentes

Module: Desarrollo Avanzado de Sistemas Multiagente



Instructor: Rubén Juárez Cádiz

¿Qué aprenderemos hoy?

-  1. ¿Por qué Python domina la IA?
-  2. Conceptos clave para desarrolladores de agentes
-  3. Entornos virtuales y dependencias
-  4. Asincronía: el superpoder de los agentes
-  5. Seguridad: manejo de API Keys
-  6. Caso práctico: Chatbot con memoria
-  7. Entregable y próximos pasos

De la idea al prototipo en horas, no en semanas

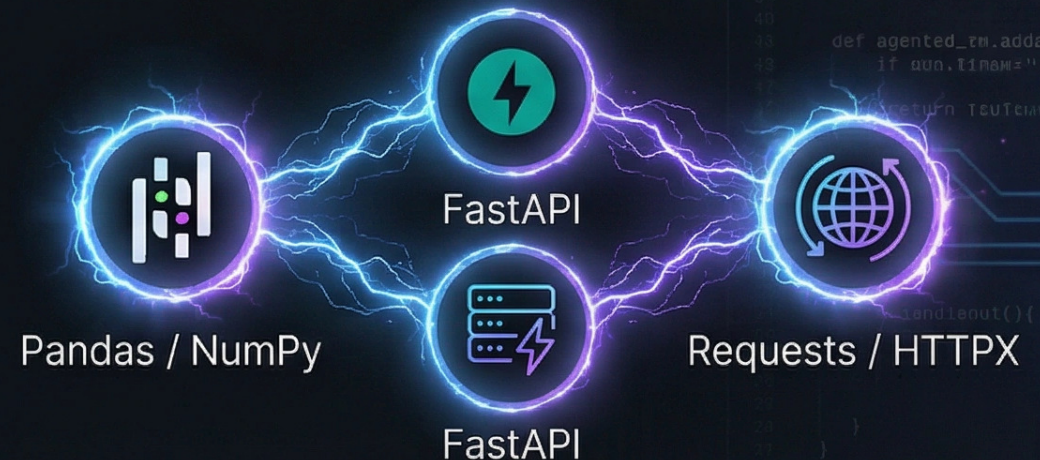
Comparativa de velocidad



Principio clave: Sintaxis similar al pseudocódigo

```
def agente_logica(datos):  
    if datos.es_valido():  
        resultado = procesar(datos)  
        return resultado  
    else:  
        return None  
# Foco en la lógica, no en la sintaxis.
```

Ecosistema integrado



venv y Conda: El primer paso antes de escribir código



¿Por qué son vitales en IA?: Las librerías de IA se actualizan constantemente (ej. LangChain v0.1 vs v0.2). Sin entornos virtuales, los proyectos se rompen.

Regla de oro: Un proyecto = Un entorno virtual. Siempre.

```
>
```

```
$ python -m venv .venv
```

```
$ source .venv/bin/activate
```

```
$ pip install openai python-dotenv
```

```
$ pip freeze > requirements.txt
```

```
>
```

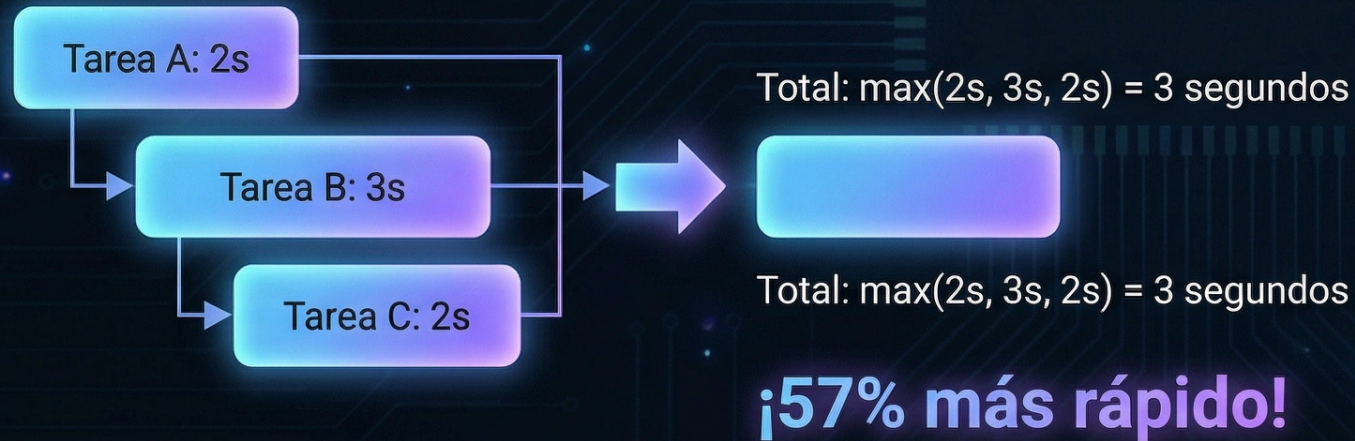

Asincronía con asyncio: El Superpoder de los Agentes

Los agentes esperan. El código asíncrono no desperdicia ese tiempo.

Sin asyncio (Secuencial)



Con asyncio (Paralelo)



- Sin asyncio (Secuencial):
 $2s + 3s + 2s = 7$ segundos
- Con asyncio (Paralelo):
 $\max(2s, 3s, 2s) = 3$ segundos
(¡57% más rápido!)

```
import asyncio

async def main():
    # Ejecución paralela
    await asyncio.gather(
        tarea_a(),
        tarea_b(),
        tarea_c()
    )

# Ejecutar
asyncio.run(main())
```


python-dotenv: Protege tus credenciales como un profesional



❌ El error más común (y peligroso):
Hardcodear la API Key en el código.

```
import openai

# ❌ INCORRECT: Hardcoded Key
openai.api_key = "sk-proj-..."

response = openai.Completion.create(
    engine="davinci",
    prompt="Translate...",
    max_tokens=60
)
```

Consecuencia: Una API Key expuesta puede generar facturas de miles de dólares.

✅ La forma correcta: Usar python-dotenv y cargar desde un archivo .env.

```
from dotenv import load_dotenv
import os
import openai

# ✅ CORRECT: Load from .env
load_dotenv()
openai.api_key = os.getenv("OPENAI_API_KEY")

response = openai.Completion.create(
    engine="davinci",
    prompt="Translate...",
    max_tokens=60
)
```

Las credenciales se mantienen seguras fuera del código fuente.

.env (NUNCA subir a Git)

```
OPENAI_API_KEY=sk-proj-...
```

.gitignore (siempre incluir)

```
.env
.venv/
__pycache__/
```


Arquitectura del Caso Práctico

Chatbot con Memoria: Entendiendo la arquitectura antes de codificar

¿Qué es la "memoria" en un LLM?

Los LLMs son stateless (sin estado).

La "memoria" se implementa enviando el historial completo de mensajes en cada llamada a la API.

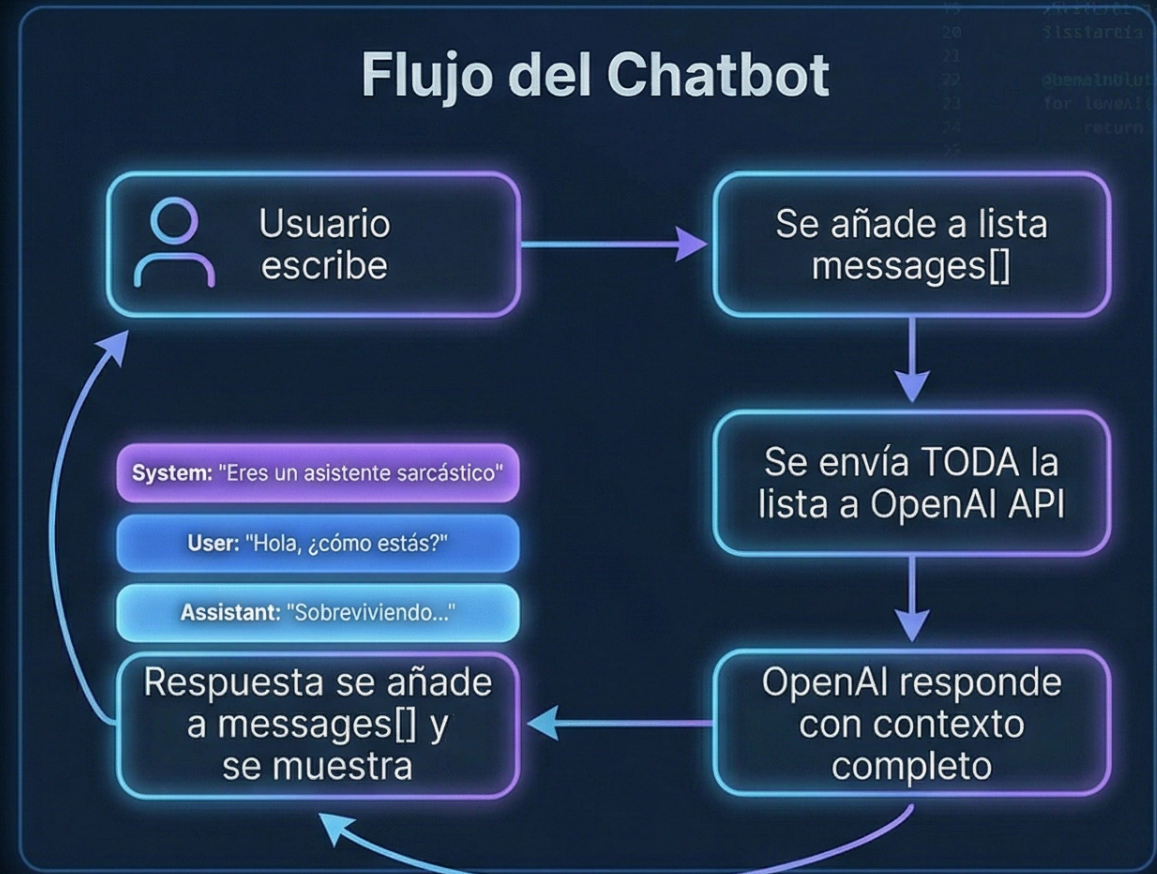
messages[]

System: "Eres un asistente sarcástico"

User: "Hola, ¿cómo estás?"

Assistant: "Sobreviviendo..."

Flujo del Chatbot



Bucle: volver al inicio

Mi primer Script Conversacional con Memoria

Menos de 30 líneas de código

- Importaciones y configuración (.env)
- Inicialización de memoria (messages = [{"role": "system", ...}])
- Bucle principal (while True)
- Captura de input y actualización de memoria
- Llamada a la API (client.chat.completions.create)

```
1 import os
2 from dotenv import load_dotenv
3 from openai import OpenAI
4
5 load_dotenv()
6 client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))
7
8 # Inicialización de memoria
9 messages = [{"role": "system", "content": "Eres un asistente útil y amigable."}]
10
11 print(";Hola! Soy tu asistente. Escribe 'salir' para terminar.")
12
13 while True:
14     user_input = input("Tú: ")
15     if user_input.lower() == 'salir':
16         break
17
18 # Actualización de memoria
19 messages.append({"role": "user", "content": user_input})
20
21 # Llamada a la API
22 response = client.chat.completions.create(
23     model="gpt-4o",
24     messages=messages,
25     max_tokens=150
26 )
27
28 assistant_response = response.choices[0].message.content
29 print(f"Asistente: {assistant_response}")
30 messages.append({"role": "assistant", "content": assistant_response})
```


Desglose del Código: Línea por Línea

Entendiendo cada componente del chatbot

Componente	Línea(s)	Descripción
Importaciones	1-6	Carga librerías y credenciales desde .env
Memoria	8-13	Define la personalidad con "system"
Bucle principal	17-35	Mantiene la conversación activa
Input	18	input() bloquea hasta que el usuario escribe
Gestión de memoria	22, 31	messages.append() acumula historial
Llamada API	24-28	Envía TODO el historial
Respuesta	30	Extrae el mensaje de OpenAI

```
1 import
2 import
3 import
4
5 import
6 import
7
8 memor
9 @
10
11
12
13
14
15
16
17 )
18
19 def
20
21
22
23
24 # ds
25 for
26
27
28
29
30
31
32
33
34
```

Puntos de extensión

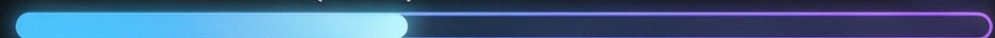
- Cambiar personalidad
- Límite de tokens
- Guardar historial en .json

Entregable y Criterios

Tu misión: Un asistente funcional en menos de **30 líneas**

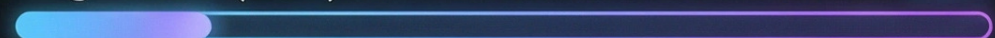
Criterios de Evaluación

Funcionalidad (40%) 40%



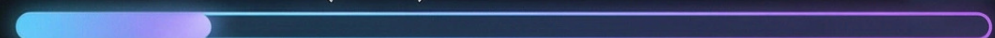
El chatbot inicia, responde y mantiene contexto

Seguridad (20%) 20%



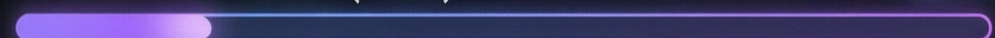
API Key cargada desde .env, no hardcodeada

Entorno virtual (20%) 20%



Proyecto con venv y requirements.txt

Personalización (20%) 20%



Personalidad del agente modificada creativamente

Entregables Requeridos

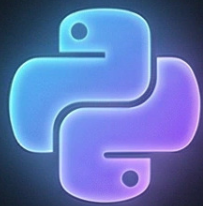
- ✓ 1. Archivo chatbot_memoria.py (máx. 30 líneas)
- ✓ 2. Archivo requirements.txt generado con pip freeze
- ✓ 3. Archivo .env.example (sin la key real)
- ✓ 4. Captura de pantalla de una conversación de prueba



Deadline: Próxima sesión del módulo

Próximos Pasos y Recursos

Python es la base. Los agentes son el destino.



Python



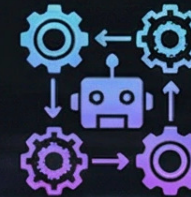
LangChain

Encadenar LLMs,
herramientas y memoria



LlamaIndex

RAG para agentes con
conocimiento propio



CrewAI / AutoGen

Sistemas multiagente
colaborativos

Recursos recomendados



Documentación oficial de OpenAI:
platform.openai.com/docs



Curso gratuito:
"Python for Everybody" — Dr. Chuck



Repositorio del módulo:
Disponible en el aula virtual

"Un agente de IA es tan bueno como el código Python que lo orquesta. Dominar Python no es opcional; es el punto de partida."

— Rubén Juárez Cádiz